

text loader

RS · 23 de septembre de 2026



In [1]:

```
from pprint import pp
from langchain_community.document_loaders import TextLoader
from pathlib import Path
```

Out [1]:

```
C:\Users\Uttam\AppData\Local\Temp\ipykernel_6972\3637659642.py:2: DeprecationWarning: `langchain-community` is being sunset and is no longer actively maintained. See https://github.com/langchain-ai/langchain-community/issues/674 for details and migration guidance toward standalone integration packages.
  from langchain_community.document_loaders import TextLoader
```

In [2]:

```
# define the path for the text file

file_path = Path("../knowledge-source/transformers.txt")

print(file_path)

print(file_path.exists())
```

Out [2]:

```
..\knowledge-source\transformers.txt
True
```

In [3]:

```
# define the loader

loader = TextLoader(file_path=file_path)
```

In [4]:

```
# Load the documents

documents = loader.load()

print(type(documents))

print(type(documents[0]))
```

Out [4]:

```
<class 'list'>
<class 'langchain_core.documents.base.Document'>
```

In [8]:

```
print(documents)
```

Out [8]:

```
[Document(metadata={'source': '..\\knowledge-source\\transformers.txt'}, page_content='# Transformer Model in Large Language Models (LLMs)\n\nThis note explains the Transformer model as used in modern Large Language Models (LLMs). It uses simple language and clear structure. You\'ll get the key ideas, components, how training and inference work, strengths and limits, and common improvements.\n\n--\n\n## 1. Big picture - Why Transformers matter\n- Transformers are the core architecture behind most modern LLMs (GPT, BERT, PaLM, LLaMA).\n- They replaced recurrent and convolutional models because they handle long-range context efficiently and are highly parallelizable on GPUs/TPUs.\n- The main innovation: self-attention, which lets the model weigh relationships among all tokens in a sequence.\n\n---\n\n## 2. Core building blocks\nA Transformer layer consists of a few repeating parts:\n\n1. Multi-Head Self-Attention\n- Computes attention scores between every pair of tokens.\n- Produces a weighted sum of token representations so each token can "look at" others.\n- Multi-head: runs several attention computations in parallel, each with different learned projections to capture different types of relationships.\n\n2. Feed-Forward Network (FFN)\n- A small neural network applied independently to each position.\n- Typically two linear layers with an activation (e.g., GELU) in between.\n\n3. Residual Connections and Layer Normalization\n- Residual (skip) connections add the input of a sublayer to its output to stabilize training.\n- LayerNorm is applied (either before or after sublayers) to improve optimization.\n\n4. Positional Encoding\n- Since attention has no sense of order, we add position information.\n- Methods: fixed sinusoidal, learned position embeddings, rotary positional encodings (RoPE).\n\n---\n\n## 3. Attention in simple terms\n- Query, Key, Value (QKV): Each token is projected into three vectors.\n- Query (Q): what the token is looking for.\n- Key (K): the token\'s content to match against queries.\n- Value (V): the content that will be combined based on matches.\n- Attention score between token i and j approximately equals softmax(Q_i · K_j / sqrt(d_k)).\n- Output for token i = sum_j (attention_weight_ij * V_j).\n- Complexity: O(n^2) in sequence length n (a key scaling challenge).\n\n---\n\n## 4.
```

Encoder vs Decoder vs Decoder-only\n- Encoder-only (e.g., BERT) – good for understanding tasks (masked language modeling).\n- Encoder-Decoder (e.g., original Transformer for translation) – encoder processes input, decoder generates output with cross-attention to encoder outputs.\n- Decoder-only (e.g., GPT) – autoregressive generation where each token attends only to previous tokens (causal masking). Common for LLM text generation.\n\n--\n\n## 5. Training objectives\n- Autoregressive / Next-Token Prediction: model learns to predict the next token given previous tokens (GPT-style).\n- Masked Language Modeling (MLM): randomly mask tokens and predict them (BERT-style).\n- Fine-tuning tasks: classification, QA, summarization, etc.\n- Instruction tuning & RLHF: specialized fine-tuning using human feedback to align models to desired behavior.\n\n--\n\n## 6. Tokenization and embeddings\n- Input text is split into tokens (subwords like Byte-Pair Encoding, WordPiece, or SentencePiece).\n- Each token maps to an embedding vector.\n- For LLMs, vocabulary size and tokenization quality affect performance and efficiency.\n\n--\n\n## 7. Inference and generation\n- Autoregressive decoding: generate tokens one at a time, feeding each predicted token back in.\n- Sampling strategies: greedy, top-k, top-p (nucleus), temperature scaling to control randomness.\n- Beam search for sequence tasks that prefer higher-probability sequences.\n\n--\n\n## 8. Scaling behavior and practicalities\n- Larger models (more layers, bigger widths) and larger datasets generally improve performance, often following approximate scaling laws.\n- Training is compute- and memory-intensive; needs parallelism: data parallelism, model parallelism, pipeline parallelism.\n- Mixed precision (FP16 or BF16) and optimized libraries help efficiency.\n\n--\n\n## 9. Limitations and risks\n- Hallucination: generating plausible but false information.\n- Biases: can reproduce harmful or biased patterns present in training data.\n- High compute and energy costs.\n- Context window limits: standard attention scales quadratically, so very long contexts are expensive.\n\n--\n\n## 10. Research directions and improvements\n- Efficient attention: sparse attention, linearized attention, sliding windows, and other approximations to reduce $O(n^2)$ cost.\n- Longer contexts: memory layers, retrieval-augmented generation, chunking strategies.\n- Better positional encodings (RoPE) and normalization techniques.\n- Modular models: mixture of experts (sparsely activated sub-networks for scaling).\n- Interpretability: probing and attention analysis to understand learned features.\n- Safety and alignment: RLHF, adversarial testing, human-in-the-loop evaluation.\n\n--\n\n## 11. Practical tips for using Transformer LLMs\n- Choose model type by task: encoder for understanding, decoder-only for generation.\n- Use appropriate tokenization: consistent with pretraining.\n- Fine-tune carefully: smaller learning rates, longer warm-up, freezing some layers sometimes helps.\n- Monitor generation: adjust temperature/top-p to balance creativity and accuracy.\n- Use retrieval when you need up-to-date or factual grounding.\n\n--\n\n## 12. Quick glossary\n- Self-Attention: letting tokens attend to each other.\n- Head: one attention computation in multi-head attention.\n- LayerNorm: normalizes features across a layer to stabilize training.\n- Residual Connection: adds layer input to output to avoid vanishing gradients.\n- Autoregressive: predicting next token conditioned on previous tokens.\n- RoPE: Rotary Position Embeddings, a popular positional method for LLMs.\n- RLHF: Reinforcement Learning from Human Feedback, used to align outputs to human preferences.\n\n--\n\n## 13. Closing summary\nTransformers power modern LLMs by enabling flexible, scalable context modeling through self-attention. The architecture is conceptually simple (attention + feed-forward layers), yet extremely powerful when scaled with data and compute. Ongoing work focuses on efficiency, longer context handling, reducing harms, and making behavior more reliable. For practitioners, understanding the core components (attention, positional encodings, tokenization, and training objectives) is enough to start using and adapting transformers effectively.\n\n--\n\nFurther reading suggestions (if you want to go deeper):\n- "Attention Is All You Need" (Vaswani et al., 2017) – original Transformer paper.\n- GPT and BERT family papers for decoder-only and encoder-only variants.\n- Survey papers on efficient attention and scaling laws.\n\nEnd of notes.\n\nnqa:# Transformer Model in LLM - 5 MCQ QA Pairs (mixed difficulty)\n\n(1) Difficulty: Easy \nQ1. What is the core architectural innovation introduced by the Transformer that replaced recurrence in sequence models? \n- A) Convolutional layers \n- B) Self-attention mechanism \n- C) Residual connections \n- D) Gated recurrent units (GRUs) \n\nCorrect answer: B) Self-attention mechanism \nExplanation: The Transformer's key innovation is self-attention, which lets each token attend to all other tokens in the sequence in parallel, eliminating the need for recurrent processing.\n\n--\n\n(2) Difficulty: Easy / Medium \nQ2. Why do Transformers use positional encodings? \n- A) To normalize activations across layers \n- B) To provide tokens with information about their order in the sequence \n- C) To reduce the model's parameter count \n- D) To initialize attention weights \n\nCorrect answer: B) To provide tokens with information about their order in the sequence \nExplanation: Self-attention is permutation-invariant, so positional encodings (sinusoidal or learned) inject sequence order information so the model can distinguish different positions.\n\n--\n\n(3) Difficulty: Medium \nQ3. In scaled dot-product attention, the dot products between queries and keys are scaled by which factor and why? \n- A) Multiply by $\sqrt{d_k}$ to increase gradients \n- B) Divide by $\sqrt{d_k}$ to prevent softmax saturation \n- C) Divide by d_{model} to regularize weights \n- D) Multiply by $1/d_k$ to speed up computation \n\nCorrect answer: B) Divide by $\sqrt{d_k}$ to prevent softmax saturation \nExplanation: The dot product magnitude grows with dimension d_k ; dividing by $\sqrt{d_k}$ keeps values in a range that prevents the softmax from having extremely small gradients and stabilizes training.\n\n--\n\n(4) Difficulty: Advanced \nQ4. What is the computational complexity (in Big-O) of a single Transformer self-attention layer with respect to sequence length n and model dimension d (assuming number of heads is constant)? \n- A) $O(n \cdot d)$ \n- B) $O(n^2 \cdot d)$ \n- C) $O(d^2 \cdot n)$ \n- D) $O(n \cdot \log n \cdot d)$ \n\nCorrect answer: B) $O(n^2 \cdot d)$ \nExplanation: Computing attention requires forming the $n \times n$ query-key similarity matrix ($O(n^2 \cdot d)$ for dot products across dimension d) and then weighted sums (also $O(n^2 \cdot d)$ in naive implementation), making overall complexity quadratic in sequence length.\n\n--\n\n(5) Difficulty: Advanced \nQ5. In autoregressive generation with a decoder-only Transformer, using key-value caching across time steps changes the per-token attention computation complexity from $O(n^2 \cdot d)$ (recomputing full attention) to what per-step complexity (for generating the next token) with current prefix length n ? \n- A) $O(n^2 \cdot d)$ – no change \n- B) $O(n \cdot d)$ – linear in current length \n- C) $O(d)$ – independent of sequence length \n- D) $O(\log n \cdot d)$ – logarithmic in length \n\nCorrect answer: B) $O(n \cdot d)$ – linear in current length \nExplanation: With caching, previously computed keys/values are reused; for a new token you compute its query and attend it to all cached keys (n) producing $O(n \cdot d)$ work per step rather than recomputing all pairwise interactions (which would be quadratic).\n\n--\n\nIf you want, I can generate additional MCQs (including math/calculation style items), or provide a small answer key sheet for quick grading.\n\n']

In [6]:

text of the document

extracted_doc = documents[0]

print(extracted_doc.page_content)

Out [6]:

Transformer Model in Large Language Models (LLMs)

This note explains the Transformer model as used in modern Large Language Models (LLMs). It uses simple language and clear structure. You'll get the key ideas, components, how training and inference work, strengths and limits, and common improvements.

1. Big picture - Why Transformers matter

- Transformers are the core architecture behind most modern LLMs (GPT, BERT, PaLM, LLaMA).
- They replaced recurrent and convolutional models because they handle long-range context efficiently and are highly parallelizable on GPUs/TPUs.
- The main innovation: self-attention, which lets the model weigh relationships among all tokens in a sequence.

2. Core building blocks

A Transformer layer consists of a few repeating parts:

1. Multi-Head Self-Attention

- Computes attention scores between every pair of tokens.
- Produces a weighted sum of token representations so each token can "look at" others.
- Multi-head: runs several attention computations in parallel, each with different learned projections to capture different types of relationships.

2. Feed-Forward Network (FFN)

- A small neural network applied independently to each position.
- Typically two linear layers with an activation (e.g., GELU) in between.

3. Residual Connections and Layer Normalization

- Residual (skip) connections add the input of a sublayer to its output to stabilize training.
- LayerNorm is applied (either before or after sublayers) to improve optimization.

4. Positional Encoding

- Since attention has no sense of order, we add position information.
- Methods: fixed sinusoidal, learned position embeddings, rotary positional encodings (RoPE).

3. Attention in simple terms

- Query, Key, Value (QKV): Each token is projected into three vectors.
- Query (Q): what the token is looking for.
- Key (K): the token's content to match against queries.
- Value (V): the content that will be combined based on matches.
- Attention score between token i and j approximately equals $\text{softmax}(Q_i \cdot K_j / \sqrt{d_k})$.
- Output for token $i = \sum_j (\text{attention_weight}_{ij} * V_j)$.
- Complexity: $O(n^2)$ in sequence length n (a key scaling challenge).

4. Encoder vs Decoder vs Decoder-only

- Encoder-only (e.g., BERT) – good for understanding tasks (masked language modeling).
- Encoder-Decoder (e.g., original Transformer for translation) – encoder processes input, decoder generates output with cross-attention to encoder outputs.
- Decoder-only (e.g., GPT) – autoregressive generation where each token attends only to previous tokens (causal masking). Common for LLM text generation.

5. Training objectives

- Autoregressive / Next-Token Prediction: model learns to predict the next token given previous tokens (GPT-style).
- Masked Language Modeling (MLM): randomly mask tokens and predict them (BERT-style).
- Fine-tuning tasks: classification, QA, summarization, etc.
- Instruction tuning & RLHF: specialized fine-tuning using human feedback to align models to desired behavior.

6. Tokenization and embeddings

- Input text is split into tokens (subwords like Byte-Pair Encoding, WordPiece, or SentencePiece).
- Each token maps to an embedding vector.
- For LLMs, vocabulary size and tokenization quality affect performance and efficiency.

7. Inference and generation

- Autoregressive decoding: generate tokens one at a time, feeding each predicted token back in.
- Sampling strategies: greedy, top-k, top-p (nucleus), temperature scaling to control randomness.
- Beam search for sequence tasks that prefer higher-probability sequences.

8. Scaling behavior and practicalities

- Larger models (more layers, bigger widths) and larger datasets generally improve performance, often following approximate scaling laws.
- Training is compute- and memory-intensive; needs parallelism: data parallelism, model parallelism, pipeline parallelism.

- Mixed precision (FP16 or BF16) and optimized libraries help efficiency.

9. Limitations and risks

- Hallucination: generating plausible but false information.
- Biases: can reproduce harmful or biased patterns present in training data.
- High compute and energy costs.
- Context window limits: standard attention scales quadratically, so very long contexts are expensive.

10. Research directions and improvements

- Efficient attention: sparse attention, linearized attention, sliding windows, and other approximations to reduce $O(n^2)$ cost.
- Longer contexts: memory layers, retrieval-augmented generation, chunking strategies.
- Better positional encodings (RoPE) and normalization techniques.
- Modular models: mixture of experts (sparsely activated sub-networks for scaling).
- Interpretability: probing and attention analysis to understand learned features.
- Safety and alignment: RLHF, adversarial testing, human-in-the-loop evaluation.

11. Practical tips for using Transformer LLMs

- Choose model type by task: encoder for understanding, decoder-only for generation.
- Use appropriate tokenization: consistent with pretraining.
- Fine-tune carefully: smaller learning rates, longer warm-up, freezing some layers sometimes helps.
- Monitor generation: adjust temperature/top-p to balance creativity and accuracy.
- Use retrieval when you need up-to-date or factual grounding.

12. Quick glossary

- Self-Attention: letting tokens attend to each other.
- Head: one attention computation in multi-head attention.
- LayerNorm: normalizes features across a layer to stabilize training.
- Residual Connection: adds layer input to output to avoid vanishing gradients.
- Autoregressive: predicting next token conditioned on previous tokens.
- RoPE: Rotary Position Embeddings, a popular positional method for LLMs.
- RLHF: Reinforcement Learning from Human Feedback, used to align outputs to human preferences.

13. Closing summary

Transformers power modern LLMs by enabling flexible, scalable context modeling through self-attention. The architecture is conceptually simple (attention + feed-forward layers), yet extremely powerful when scaled with data and compute. Ongoing work focuses on efficiency, longer context handling, reducing harms, and making behavior more reliable. For practitioners, understanding the core components (attention, positional encodings, tokenization, and training objectives) is enough to start using and adapting transformers effectively.

Further reading suggestions (if you want to go deeper):

- "Attention Is All You Need" (Vaswani et al., 2017) – original Transformer paper.
- GPT and BERT family papers for decoder-only and encoder-only variants.
- Survey papers on efficient attention and scaling laws.

End of notes.

qa:# Transformer Model in LLM - 5 MCQ QA Pairs (mixed difficulty)

1) Difficulty: Easy

Q1. What is the core architectural innovation introduced by the Transformer that replaced recurrence in sequence models?

- A) Convolutional layers
- B) Self-attention mechanism
- C) Residual connections
- D) Gated recurrent units (GRUs)

Correct answer: B) Self-attention mechanism

Explanation: The Transformer's key innovation is self-attention, which lets each token attend to all other tokens in the sequence in parallel, eliminating the need for recurrent processing.

2) Difficulty: Easy / Medium

Q2. Why do Transformers use positional encodings?

- A) To normalize activations across layers
- B) To provide tokens with information about their order in the sequence

- C) To reduce the model's parameter count
- D) To initialize attention weights

Correct answer: B) To provide tokens with information about their order in the sequence

Explanation: Self-attention is permutation-invariant, so positional encodings (sinusoidal or learned) inject sequence order information so the model can distinguish different positions.

3) Difficulty: Medium

Q3. In scaled dot-product attention, the dot products between queries and keys are scaled by which factor and why?

- A) Multiply by $\sqrt{d_k}$ to increase gradients
- B) Divide by $\sqrt{d_k}$ to prevent softmax saturation
- C) Divide by d_{model} to regularize weights
- D) Multiply by $1/d_k$ to speed up computation

Correct answer: B) Divide by $\sqrt{d_k}$ to prevent softmax saturation

Explanation: The dot product magnitude grows with dimension d_k ; dividing by $\sqrt{d_k}$ keeps values in a range that prevents the softmax from having extremely small gradients and stabilizes training.

4) Difficulty: Advanced

Q4. What is the computational complexity (in Big-O) of a single Transformer self-attention layer with respect to sequence length n and model dimension d (assuming number of heads is constant)?

- A) $O(n \cdot d)$
- B) $O(n^2 \cdot d)$
- C) $O(d^2 \cdot n)$
- D) $O(n \cdot \log n \cdot d)$

Correct answer: B) $O(n^2 \cdot d)$

Explanation: Computing attention requires forming the $n \times n$ query-key similarity matrix ($O(n^2 \cdot d)$ for dot products across dimension d) and then weighted sums (also $O(n^2 \cdot d)$ in naive implementation), making overall complexity quadratic in sequence length.

5) Difficulty: Advanced

Q5. In autoregressive generation with a decoder-only Transformer, using key-value caching across time steps changes the per-token attention computation complexity from $O(n^2 \cdot d)$ (recomputing full attention) to what per-step complexity (for generating the next token) with current prefix length n ?

- A) $O(n^2 \cdot d)$ – no change
- B) $O(n \cdot d)$ – linear in current length
- C) $O(d)$ – independent of sequence length
- D) $O(\log n \cdot d)$ – logarithmic in length

Correct answer: B) $O(n \cdot d)$ – linear in current length

Explanation: With caching, previously computed keys/values are reused; for a new token you compute its query and attend it to all cached keys (n) producing $O(n \cdot d)$ work per step rather than recomputing all pairwise interactions (which would be quadratic).

If you want, I can generate additional MCQs (including math/calculation style items), or provide a small answer key sheet for quick grading.

In [9]:

```
# metadata of document
```

```
pp(extracted_doc.metadata)
```

Out [9]:

```
{'source': '..\\knowledge-source\\transformers.txt'}
```

In [10]:

```
# number of documents
```

```
len(documents)
```

Out [10]:

```
1
```